

Security Baseline (MVP)

Purpose

Define the minimum security posture for a production MVP.

1. Authentication

- JWT access token for API access.
 - Passwords must be hashed (bcrypt/argon2).
 - Access tokens must be short-lived (e.g., 15m).
 - Refresh tokens (optional MVP) stored securely if implemented.
-

2. Authorization (RBAC)

Roles (MVP):

- **ADMIN** (Owner)
- **CASHIER**
- **WAITER**

Rules:

- Waiter can create/update orders assigned to tables.
 - Cashier can close orders and manage payments.
 - Admin can manage menu, users, tables, and view reports.
-

3. Input Validation

- Use DTO validation on controllers (class-validator).
 - Sanitize and validate:
 - phone numbers
 - addresses
 - product quantities
 - price fields (never trust client totals)
-

4. Secrets Management

- Never hardcode secrets in code.
 - Use environment variables:
 - DB connection string
 - JWT secret
 - Siigo credentials (future)
 - WhatsApp token (future)
-

5. CORS

- Only allow known origins (Public Web domain and Internal App domain).
 - Restrict methods and headers.
-

6. Rate Limiting

- Basic rate limiting at Nginx (recommended).
 - Optional API rate limiting via NestJS throttler.
-

7. Security Headers (Nginx)

- HSTS
 - X-Frame-Options
 - X-Content-Type-Options
 - Referrer-Policy
 - Content-Security-Policy (basic in MVP)
-

8. Database Security

- Use AWS RDS.
 - Restrict inbound access to EC2 security group only.
 - Apply migrations through CI/CD or controlled process.
 - Backups enabled.
-

9. Logging & Audit

- Log authentication events (success/failure).
 - Order lifecycle changes should store:
 - created_by, updated_by
 - timestamps
 - status history (optional but recommended)
-

10. Threats & Mitigations (MVP)

- SQL Injection: use Prisma parameterization (no raw queries without escaping).
- Broken Access Control: enforce RBAC checks in use cases.
- Secrets leakage: .env never committed; use GitHub secrets.
- DoS: rate limits + security groups.